

1 Question

- (a) **Argue that this is a correct algorithm for the Independent Set problem on graphs of maximum degree three**

We prove that the provided algorithm is correct by using a proof by exhaustion. Let us first consider the base cases.

If G has a node of degree 0, then, by definition, it has to be included in the independent set, as v is adjacent to no other vertex. Note that this rule is only here for completeness; A valid independent set could be found by ignoring vertices of degree 0. However, in that case, the independent set would not be a maximum.

If G has a node of degree 1, then this node is only connected to one other node, which we denote as u . Therefore, to compute a maximum independent set, either v or u , but not both, need to be included in the independent set. Because including u and v in the independence set depends on the other nodes in the graph, we have to first solve the independence set of the graph G without $\{u, v\}$, and then, if any neighbors of u is in the independent set, we can add v to the independent set. Otherwise, we can add either v or u to the independent set.

If G has a vertex of degree 2 with both neighbors being adjacent, then by adding v to the independent set, both neighbors are covered, and can therefore be ignored for consideration in the independent set. Therefore, recursively solving the subset without v and its neighbors will yield a correct solution.

If G has a vertex v of degree 2, and that this vertex is adjacent to another vertex w of degree 2, then the simplification rule can be applied to find a correct solution. Because $v - w - y$ form a chain, either v or w will be included in S . Furthermore, because v has another neighbor x , and that therefore the chain $x - v - w - y$ exist. There are now multiple possibilities: either x is included in S , or one of its neighbors, which is either v or another neighbor of x . If $x \in S$, then we can add w iff $y \notin S$. If $x \notin S$, then we have to add v to S . The same argument can be repeated for y . Therefore, removing $\{v, w\}$ from the graph and adding an edge between x and y will yield a correct solution, as the maximal independent set will try to cover x and y , and, depending on whether x , y , or their neighbors are included in S , we can add v or w to S .

If all nodes are of degree 2, then the solution is easy to compute, as the graph forms a chain that is connected to form a loop.

For nodes with degree 3, we need to consider all possible solutions exhaustively. Hence, we can recurse 1 time with just v removed from the graph (which imply $v \notin S$), and a second time with v and its neighbors removed from the graph, which would imply $v \in S$. The returned solution with the largest set is the best solution, and therefore the algorithm is correct.

Because all nodes have a degree of at most 3, no other cases can exist. Therefore, the algorithm is correct and finds an independent set of the largest size.

(b) **How do we solve the problem if all vertices have degree exactly two?**

A graph where every node has degree 2 forms a chain that is connected such that it forms a "loop". Thus, starting from any node, we can add it to the independent set, and remove its neighbors. Then, we have exactly two nodes with degree 1. We can then add one of the nodes with degree 1 to the independent set and remove it and its neighbors from the graph. Because in a graph with $n \geq 3$ nodes, this last step is being repeated $\frac{n-3}{2}$ times, we have an algorithm that runs in linear time.

Note that n cannot be less than 3, as then each node can have a degree of at most $n - 1$, and with $n = 2$, we have a maximal degree of $2 - 1 = 1$.

(c) **Let G be a graph with maximum degree three, and n_3 vertices of degree three. Let v be a vertex of degree three. Let H be obtained from G by removing v , and then applying all reduction rules of the algorithm exhaustively (all if cases, but not the otherwise case). Argue that H has at most $n_3 - 4$ vertices of degree three.**

Let us assume that G is connected, as we can simply repeat the process for each of the subgraphs.

By applying all the "if" rules exhaustively and recursively, we obtain a graph H where every node is of degree at least 2. Furthermore, we can observe that every node of degree 2 has exactly two neighbors with degree 3 that are not adjacent, as otherwise, the node would have been removed by an "if" rule.

Let us now remove an arbitrary node v with degree 3. Now, each of the neighbors of v have one less degree. There exist two cases for each of the neighbors of v : either they have degree 2, or they have degree 1.

If a neighbor u of v has degree 1 after removing v , then u can be removed from one of the "if" rules. Because every node of degree 2 has two neighbors of degree 3, we know that, after removing u , the neighbor w (with $w \neq v$) has degree 2, and therefore reduces n_3 by 1.

Let u be an arbitrary neighbor of v with degree 2 after removing v . There exist two cases for u : (1) u is adjacent to another node w with degree 2 or (2) the neighbors of u are adjacent.

Let us consider the first case. Let us denote z as the neighbor of u with degree 2. Then, we know that the other neighbor of z has degree 3. Because the other neighbor of u has either degree 2 or 3, we can recursively apply the "if" rule, removing $\{u, z\}$ from the graph and connecting the two neighbors with an edge. This can be done until we connect two nodes with degree 3, or a node with degree 2 to a node with degree 3, where the node with degree 2 has no other neighbor with degree 2. Because u was a node with degree 3 in G , this process reduces n_3 by at least 1.

Let us now consider the second case. In that case, we can apply the "if" rule removing u and its neighbors. Because both neighbors of u have degree 3, we know that we can reduce n_3 by at least 2, because u is adjacent to at least one node with degree 3, and removing u and that node reduces n_3 by 2. Both neighbors of u cannot have degree 2, as otherwise they would be removed by an "if" rule and replaced by an edge.

Because removing v reduces n_3 by 1, and each of the three neighbors of v also reduce n_3 by at least 1 by

using the "if" rules, we know that, from deleting v , we reduce n_3 by at least 4. We then have at most

$$n_3 - 4$$

nodes of degree 3 after removing v and applying all "if" rules exhaustively.

- (d) **Let G be a graph with maximum degree three, and n_3 vertices of degree three. Let H' be the graph, obtained from G by removing v and its neighbours. Argue that H' has at most $n_3 - 4$ vertices of degree three.**

We assume the same conditions as in the previous question.

We remove v and its three neighbors from G , and apply all the "if" rules exhaustively. We know that each neighbor of v has at least degree 2 or 3 after applying the "if" rules.

If a neighbor u of v has degree 2 after removing v , then we know that, by removing u , we reduce n_3 by at least 1. We then only have to consider the case where a neighbor u of v has degree 1 after removing v .

It follows that u is removed from the graph from the second "if" rule. Because, before removing v but after applying the "if" rules, each node of degree 2 is connected to two nodes of degree 3, by removing u , a node with degree 3 loses one neighbor, and therefore this process reduces n_3 by at least 1.

Therefore, because each of the three neighbors of v induces a reduction of n_3 by at least 1 each, we know that, after removing v and its neighbors, and applying all the "if" rules exhaustively, we have at most $n_3 - 4$ nodes of degree 3 in H' .

- (e) **Show that the algorithm uses $\mathcal{O}^*(2^{n/4})$ time.**

Because G has a maximum degree of 3, we know that each of the rules can be run in constant time on each nodes. This is because, in the worst case, we have to check that three neighbors of a node are (not) adjacent, and because, in the worst case, each neighbor has degree 3, we have to check 3^3 nodes, which can therefore be done in $\mathcal{O}(1)$. Note that, to check if a rule can be applied, we still have to loop over all n nodes, and therefore takes $\mathcal{O}(n)$ time.

This means, that we can compute H and H' (from iii and iv), in polynomial time, as we only have to apply the "if" rules exhaustively and recursively. Because each rule removes at least one node, this process terminates in polynomial time.

Therefore, after applying all the "if" rules in polynomial time, we have to "otherwise" rule, which tests whether including a node v with degree 3 in S is better than to exclude it. To that end, the algorithm bruteforces the best solution for every combination of subset of nodes with degree 3. This would take $\mathcal{O}^*(2^{n_3})$ time. However, because applying one of the rules of the otherwise rule reduces n_3 by at least 4, we can prune the bruteforce to $\mathcal{O}^*(2^{n_3/4})$ time. Because $n_3 \leq n$, we then have $\mathcal{O}^*(2^{n/4})$.